Exploring the Capabilities and Constraints of AI and Large Language Models in Game Design Methodology

Authored by Brock Allan

Advised by Chancellor's Professor Cristina Videira Lopes, Department of Informatics, University of California Irvine

Spring 2026

Abstract

This paper explores the question of how developed Artificial Intelligence is in the field of generating code for use in game design. Whether or not AI has the current capability of replacing designers will be evaluated using multiple levels of testing and comparative analysis of AI-generated and human-designed outputs. In order to compare the two, we design a scoring mechanism that establishes a clear basis for comparison on what it means to be a game designer. Using our scoring mechanism, we run a multistage test using prompt engineering across two different models to understand how much a Large Language Model will generate given certain support and how generation has improved over time. The prompts provided will be of different depth to provide fairness for the prompting of the model, giving it a chance or enough parameters to understand what it needs to do without human bias. Using our scoring mechanism, the paper establishes a clear distinction in the quality of work between AI generated content and human written code. The key metrics underlying this assertion involve an understanding of what contributes to an engaging video game, as well as a comprehension of how different video game genre's function. An additional important metric is the quality of the codebase itself, as video games require fast and efficient code that is written in clear, concise, and well-structured ways to ensure both usability and maintainability for developers.

Table of Contents

Introduction

Video games represent one of the most complex and interdisciplinary forms of digital modern media. They combine computational systems, interactive design, narrative structure, visual art, audio engineering, and user psychology into a single playable experience. At their very core, video games are rule-based digital systems in which players interact with programmed environments to achieve goals, make decisions, and receive feedback. Unlike passive media, games require active participation, creating dynamic feedback loops between player input and system response. Traditional video game development is inherently iterative. Designers prototype mechanics, test systems, gather feedback, rebalance features, and refine gameplay through repeated cycles of evaluation. In 2026, Large language models and generative AI systems can now assist in creating mechanics, narrative structures, dialogue systems, and even complete playable prototypes. The effectiveness of AI, however, depends heavily on how it is guided and programmed. Prompt engineering techniques such as zero-shot, single-shot, and few-shot determine the quality, coherence, and playability of AI-generated content.

This project explores the role of AI in video game development by systematically evaluating how different prompting methodologies influence the quality of generated games. Through a series of structured AI trials and human-AI collaborative design, this study examines whether increasingly refined prompting strategies produce more functional and engaging video games. To ensure no objective bias during evaluation, a scoring mechanism is introduced to assess AI-generated games across key design principles such as Clarity, Flow, Balance, Length, Integration, and Fun. The output of each model is also evaluated on the quality of its codebase using the four pillars of programming. In comparing multiple AI-generated case studies under

consistent evaluation criteria, this research aims to determine how AI can best serve as a tool, not as a replacement for human designers.

What is a Video Game

Video game development cannot be discussed without understanding what a video game is in its essence. Scholars have concerned themselves with potential definitions of what a video game is for some time, for example: “A videogame is a game which we play thanks to an audiovisual apparatus and which can be based on a story” (Esposito 2005). Now of course there are several things wrong with this definition. A video game does not need audio to be played so “audiovisual” does not make sense here. A video game also does not need a story to be fun or played such as Tetris (1988) or 2048 (2014). The addition of these features certainly helps to improve the overall experience but is not required to play an actual game. Researching this field has shown that scholars fail to take the word “video game” at its face value and try to string together a definition based on what they think it is rather than what its stated to be, for example: “X is a videogame if it is an artefact in a digital visual medium, is intended primarily as an object of entertainment, and is intended to provide such entertainment through the employment of one or both of the following modes of engagement: rule bound gameplay or interactive fiction” (Tavinor 2008). There is an important topic in the study of programming languages called syntax and semantics where syntax is the form of its expressions, statements, and program units. Essentially Syntax is how a language is written and the rules that coincide with its definition. Semantics is the meaning of those expressions, statements, and program units. Semantics is therefore how the language functions given a string of syntactically correct statements. Syntax and Semantics are closely related, such that good syntax should explain the semantics of a language. Scholars tend to try to define a video game by just using its semantics, how it functions, without looking at its syntax, how it's stated.

Essentially a video game boils down to just two words, “video” and “game,” and these are the only two words that we should be concerning ourselves with since good semantics should be explained by their syntax. Video is an electronic medium for recording, reproducing, broadcasting, and displaying moving visual images across digital and analog platforms. Game is a form of play or sport for entertainment, played according to a set of rules to achieve a specific goal. The result is often decided by skill, strength, or luck. Breaking down video games into its essence now allows us to create the following strict definition. “A video game is a medium that is played on a media device through a controlling apparatus to achieve some goal.” Breaking down this definition, we can explain it as a medium (a material or form) that is specifically played instead of watched on a device that showcases some visual form controlled by a device for a specific purpose.

Role of AI in Game Design

When considering the use of artificial intelligence in video games, many immediately think of NPCs, also known as Non-Player Characters. These characters navigate environments, react to player input, and simulate intelligent behavior. As Filipović notes, “the most obvious use of AI in games is in controlling NPCs,” with additional applications including scripting and pathfinding systems that determine how characters move through terrain and obstacles (Filipović 2023). Due to recent advances in generative models in the last 10 years, AI’s role now reaches outside the realm of just NPCs. “AI has an essential role in the development and testing of video games,” as it can “automate tedious tasks such as bug detection and gameplay testing, saving time and developer resources” (Filipović 2023). Beyond automation, AI can also be central to the design of gameplay itself, treating AI as a core mechanic rather than a background utility. This perspective transforms AI from a purely technical tool into a design lever, influencing level design, rule systems, and player interaction loops. In this way, “AI extends beyond procedural generation or

testing, becoming an integral part of both the creative and strategic responsibilities of the game developer" (Treanor et al. 2015). AI is currently transforming the game design industry by shifting from manual scripting to AI-assisted coding and automated development pipelines. Since 2025, over "50% of game development companies have already integrated AI into their workflows" to reduce production time and costs (CAPITOL 2025). Developers utilize these tools to handle repetitive tasks, such as writing test cases or basic event listeners. Some models, such as Microsoft's Copilot, even act as an AI pair programmer inside your editor, providing "real-time, context-aware suggestions ranging from single-line completions to complex algorithmic blocks and automated debugging" (Friedman 2021). AI-assisted coding now accounts for a significant portion of boilerplate generation, sections of code that are repeated in multiple places with little to no variation. This has further been expanded into Prompt-Based Development, where developers provide "high-level functional descriptions to generative models to facilitate rapid prototyping and the immediate synthesis of foundational features" (Dohmke 2023). The primary focus of this research will center on this interaction, the efficiency of natural language prompts in the generation of functional code bases.

Video Game Development Methodology

A methodology is a body or set of rules and ideas that are important in science, art, or discipline, specifically the analysis of the principles or procedures of inquiry in a particular field (Methodology n.d.). Methodology when relating to game design is therefore the steps and ideas related to each concurrent step in the design process, why they exist, and how a finished product is produced with these steps. Modern game development methodologies are characterized by an iterative process with an agile methodology. Across different methods, generally 7 steps persist over multiple models as seen on figure 1. These steps are Feasibility study, Requirement Analysis,

Design, Implementation, Testing, Deployment, and Maintenance. Feasibility study is where the design team decides the economic considerations with the project and if they are feasible under time constraints. Requirement Analysis is where the team forms the technical and personnel requirements for the project including resource allocation. The design step is where the team documents how the project will work, how it operates, and how it's incrementally developed. Implementation is the stage at which the "development team uses the design that was produced to implement the software using coding language or other tools" (Harman 2023). Testing is where the team tests software for completeness and correctness, this also includes checking for edge cases or serious bugs. The Deployment stage is where the product is deployed for use for the public or private consumer. Finally, the Maintenance step is where the software is continuously provided support and patches overtime to ensure operability.

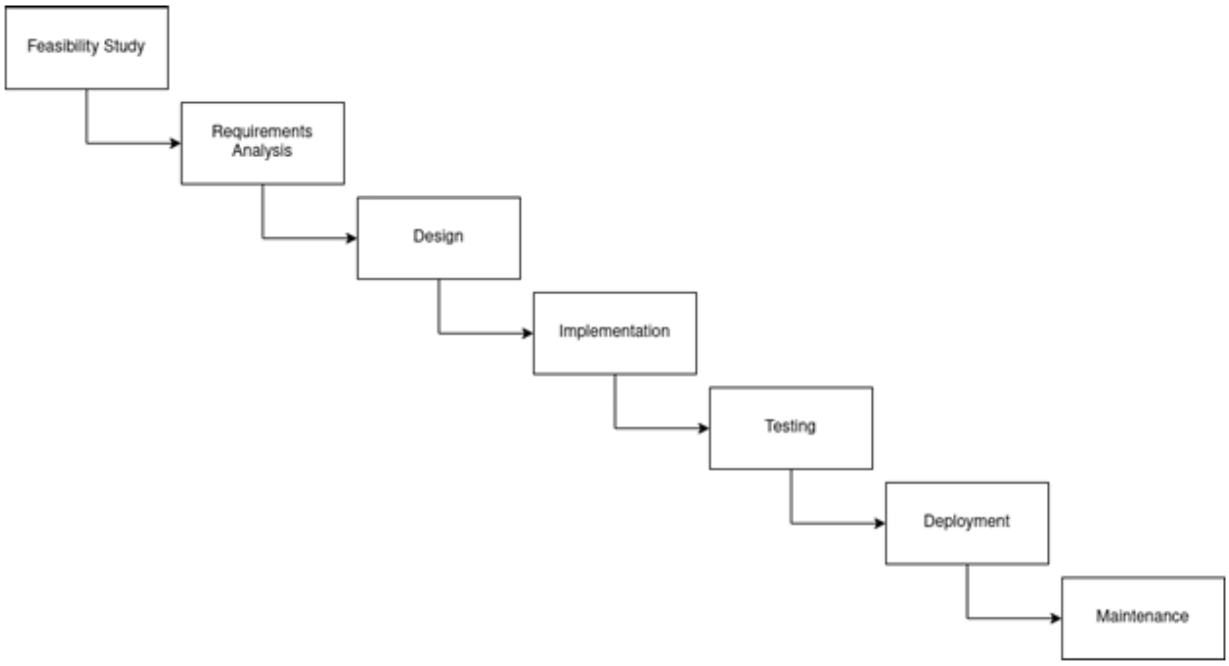

Figure 1: General Steps of Video Game Development Methodology (Harman 2023)

Specifically, we will be taking a closer look at the first 5 steps, feasibility through testing, rather than the whole process when training our AI models since we won't be deploying anything.

Modern game development uses a Spiral model which is a model that incorporates repetition and iteration as opposed to a linear model like Waterfall. “The spiral model gives a framework for choosing what to do next but does not directly prescribe what must be done and in what order” (Harman 2023). The spiral portion of the model is similar to what is called a “Sprint” where meetings are planned after each iteration to discuss progress and what to fix. The major benefit of this model is that it allows for continuous refinement to meet customer requirements and offers greater flexibility than a classic waterfall model. Earlier stages of our experiment will use a more linear model such as Waterfall as they will be more direct with prompting. Later stages of our experiment will make use of the spiral model as it supports more feedback and discourse between a user and the LLM, we can see the difference between the two models in figure 2.

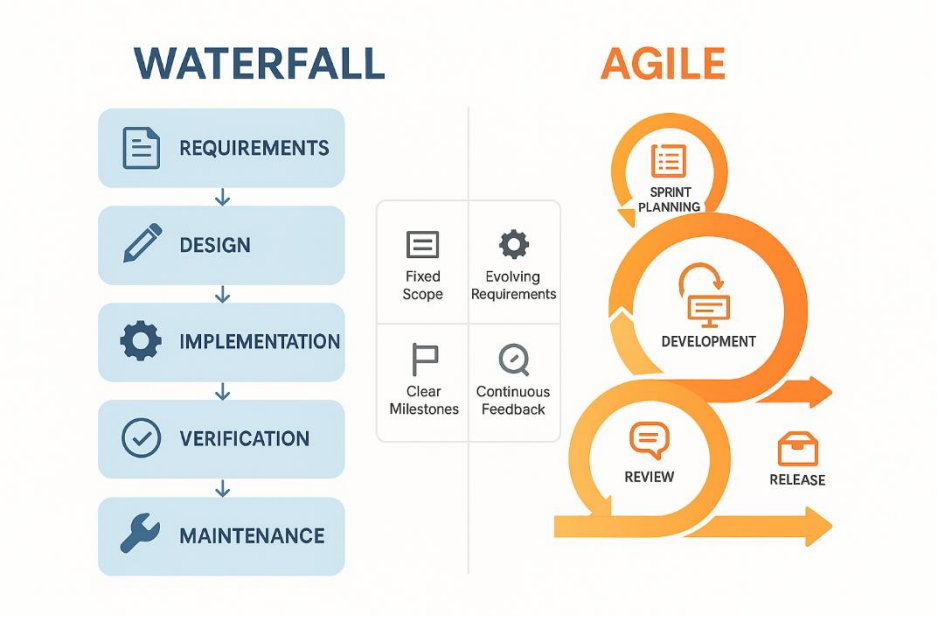

Figure 2 Waterfall and Agile Models (PM 2025)

This experiment is conducted within the Scrum framework, an agile methodology designed for managing complex, adaptive problems as seen in figure 3 (Schwaber & Sutherland 2020). By breaking the research into distinct Sprints, each trial serves as a structured window to test specific

prompting depths. Scrum can be broken up into 3 categories: Roles, Artifacts, and Stages. The roles at play are Product Owner, Scrum Master, and Development Team who defines the vision of the project, facilitates it, and performs work in it. The Artifacts are the work itself being composed of a Product Backlog, Sprint Backlog, and Increment, representing the main goal, specific tasks, and finished product. The Stages are where Inspection and Adaptation occur known as Sprints, Sprint Reviews, and Sprint Retrospectives (Schwaber & Sutherland 2020).

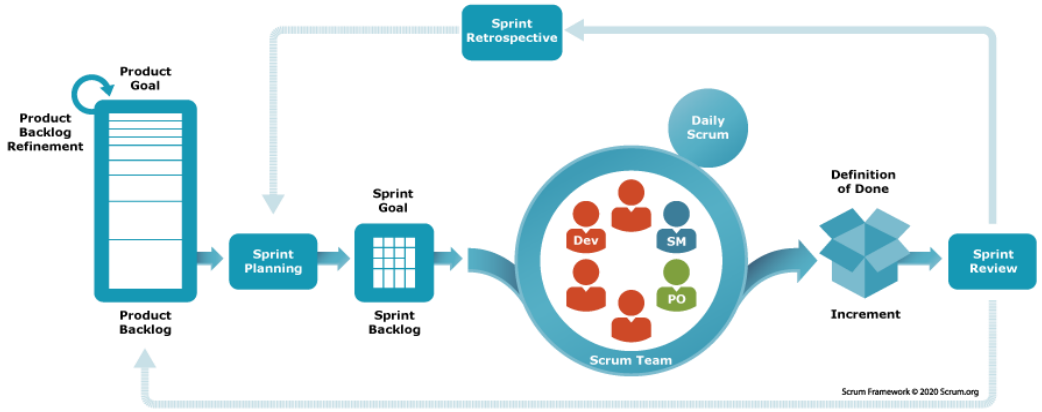

Figure 3 Scrum Framework (Scrum 2026)

The researcher serves as the Product Owner, responsible for defining the "Vision" of the experiment, maintaining the research objectives, and establishing the scoring mechanism for evaluating outcomes. The Development Team is a collaborative unit consisting of the Large Language Model (placeholder) acting as the primary code generator, while the researcher serves as the lead debugger and prompt engineer. The Product Backlog is primarily the generation of a functional "Grape Picking" game utilizing varied prompting depths. The Sprint Backlog focuses on a designated prompting strategy per trial. The Increment is the final Python files generated at the conclusion of each trial representing the potentially shippable product scored against our mechanism. Inspection occurs during the Sprint Review phase at the end of each game trial, where

the code is analyzed for metrics such as syntactic noise and mechanical coherence. Adaptation is conducted during the Sprint Retrospective, where the prompting strategy is refined for the subsequent iteration based on the failures or successes of the previous model. Figure 4 outlines the experimental roadmap for the four case studies. Each trial represents a distinct Sprint, with escalating complexity in the Strategy column. If the Inspection of Game 1 reveals high syntactic noise, the Adaptation for Game 3 moves toward Contextual Alignment through Single/Few-Shot prompting. This ensures that each Sprint builds upon the technical spikes of the previous trial.

Trial	Sprint Goal	Strategy	Expected Outcome
Game 1 & 2	Baseline "Raw" Capability	Zero-Shot	A simple, likely buggy, Python script.
Game 3 & 4	Contextual Alignment	Single/Few-Shot	A functional "Oregon Trail" clone with GUI.
Game 5 & 6	Iterative Refinement	Spiral / "Simulated Tuning"	High-quality code with error handling.
Game 7	Human-Centric Design	Pair Programming	Optimized, "clean" code ready for a player.

Figure 4: Sprint Objectives and Prompting Strategies

Prompt Engineering for AI Game Development

Prompt Engineering is the practice of designing and refining prompts or queries given to a large language Model (LLM) to generate accurate, relevant, and useful output. “Prompt Engineering is the means by which LLMs are programmed via prompts, where a prompt is a set of instructions provided to an LLM that programs the LLM by customizing it and/or enhancing or refining its capabilities” (White et al. 2023). The quality of the output is dependent on the clarity, structure, and level of context provided in the prompt. Prompt engineering techniques can be categorized based on how much context and guidance are given to the model.

Zero-shot prompting is a technique where the model is given a task description in natural language but no explicit examples of the task. The model must rely “purely on its pre-trained knowledge” when generating a response (Brown et al. 2020). AI Games 1 and 2 below follows this prompting method; the prompt used for this was:

"I want you to code me a game in python about grape picking."

As you can see there was no context or examples provided for the prompt. The model was provided minimal direction or constraints, and a severe lack of design specification. While Zero-shot prompting can produce usable results, the output may not align exactly with what the user was wanting from the model. Zero-shot in this regard "often struggles with tasks requiring specific domain adaptation or fine-grained reasoning, as the model lacks the 'in-context' demonstrations necessary to resolve semantic ambiguity" (Brown et al. 2020).

Single-shot refers to a prompt that includes a single detailed example of how to generate output, "which serves as a frame of reference for the model's generation process" (Brown et al. 2020). The following is an example used in AI Games 3 and 4 to generate the initial code for the trial.

"Im going to take you through the game development process to develop a game using python and the pygame library. To start our idea will be a game about "grape picking." We will be developing this game as a text based game with a simple graphical interface. Our game will look similar to that of the Oregon trail game where a person can choose from multiple options to certain scenarios and progress through the story to eventually survive."

This increased context allows the model to generate a more targeted and structured result compared to Zero-shot prompting, as it aligns the model's pre-trained knowledge with a specific genre and mechanical framework. Few-shot prompting builds upon the Single-shot method by providing a small set, typically 2-5, of examples or "iterative constraints" (Brown et al. 2020). Below is an example from AI Games 3 and 4.

“ Certain options will lead to random events that could cause a game over for the player. For simplicity we will start with a version of the game that creates a GUI that is just text and buttons that represent options. Remember to stay on theme about grape picking, which can then lead to some adventure. I need you to generate all python files that will be helpful in displaying the gui, all the text, and all paths that the game can follow. An example of how one would do something like this is a series of if statements that create branching paths and display text based on what path we are in.”

Simulated Tuning in this experiment is known as Chain-of-Hindsight or Self-Correction, essentially “Prompt-based Fine-Tuning,” where the model's behavior is narrowed by providing positive and negative constraints of what to and not do (Liu et al. 2023). Normally Fine-Tuning refers to changing a model's weights or hyperparameters to adapt to a new task. Simulated Tuning or “Fixed-LM Prompt Tuning" freezes the models’ parameters and the tuning happens “entirely by optimizing the prompt itself” (Liu et al. 2023). Below are a few short examples from AI games 5 and 6.

“We are at the concept and ideation stage of game development, I want you to discuss with me ideas about how to make a game about grape picking.“

“We are now in stage 2 of game development: Assets and Environment creation. I want you to think about and implement some assets or environments that you can use to create this game revolving around the ideas you produced in step 1.”

“We are now in stage 7: testing , QA, and Optimization. Play testers have stated that the dialogue and story promised has not been delivered.”

The tuning occurs entirely within the context window by providing a sequence of Hindsight Feedback, which are direct critiques of sub-optimal outputs that force the model to self-correct in real time (Liu et al. 2023; Madaan et al. 2023). This additionally follows the "Self-Refine framework", where the model acts as both the Generator and Feedback Provider. In the experiment, this is the stage where we prompt the model to think about and implement assets based on previous ideation. The model tunes its next output based on the successes or failures of its previous stages (Madaan et al. 2023).

Creating an AI Scoring Mechanism

The evaluation metrics will take two forms: evaluation of the generated code itself, and the playability of the resulting game. Tulqin o'g'li in "Rubric Design for Grading Programming Assignments" claims that Designing a rubric for a programming assignment typically begins by identifying the following core dimensions: Correctness/Functionality, Efficiency, Style/Readability, Documentation, and Algorithmic Complexity. These dimensions are very similar to the four pillars of code quality: Correctness, Maintainability, Readability, and Efficiency. The four pillars are an established professional standard in software engineering that are formally codified in academic benchmarks and foundational texts. The RACE benchmark is a multi-dimensional framework that was specifically developed to evaluate the quality of code generated by Large Language Models (LLMs) across these exact four dimensions (Wang et al., 2026). Taking inspiration from Tulqin o'g'li, Wang, and the four pillars we have composed the following rubric, which uses the same four pillars but with slightly different definitions:

Functionality: Does the code run without errors and fulfill all requirements. Does the code generated work straight away without minor modifications. Does the generated code correctly solve the proposed issue or requirement.

This metric evaluates the autonomy and accuracy of the generated code. It measures whether the output is immediately executable and if it maintains strict adherence to the provided specifications. Code is successful if and only if it resolves the target issue without necessitating human intervention or "minor modifications," ensuring the model has fully captured the underlying logic of the requirement.

Efficiency: Does the solution use data structure, libraries, and algorithms effectively without unneeded complexity? Does the code run in a reasonable amount of time? Does the code use a minimal amount of memory? Does the generated code perform its task using resources that it did not need to complete it?

This metric assesses the algorithmic complexity of the generated solution. It evaluates whether the model utilizes appropriate data structures and libraries to minimize time complexity and spatial overhead or space complexity. A solution is deemed efficient if it performs the required task without "resource leakage" or unneeded complexity, ensuring that the code remains optimal under standard constraints and does not consume unnecessary system memory.

Readability: Is the code well organized and includes well described comments? Does the code use abstraction through functions or classes? Are there consistent and meaningful naming conventions? Is the codebase indented and uses short line lengths? Are complex algorithms well defined and explained?

This metric evaluates the communicative quality of the generated codebase. It assesses whether the code adheres to professional legibility standards and measures the model’s ability to utilize architectural abstraction to provide contextual documentation for complex logic. A high

readability score indicates that the code is not only functional but also "self-documenting," significantly reducing the cognitive load required for peer review or long-term maintenance.

Maintainability: Is the code written in such a way that it can be reused with new additions? Will additional code break systems already in place? Does the solution avoid monolithic structures by using modular functions or object-oriented principles? Does the codebase allow room for future changes or does it hardcode certain aspects?

This metric evaluates the future proofness of the generated code. It assesses whether the solution utilizes modular design patterns to prevent large dependency chains. A highly maintainable solution avoids hardcoded values in favor of configurable parameters, ensuring that the codebase can be reused or expanded without inducing "regressive failures." This dimension measures the model's ability to produce code that is not just a one-time fix, but a sustainable foundation for iterative development.

Each metric described above will be scored on a 4 point scale where 1 represents "Unsatisfactory," 2 is "Needs improvement," 3 is "Satisfactory", and 4 is "Exemplary." The following figure 5 goes into detail of what each point on the scale represents.

Metric	1 Unsatisfactory	2 Needs Improvement	3 Satisfactory	4 Exemplary
Functionality	Code fails basic requirements; contains syntax/runtime errors.	Functional but fails edge cases; requires manual modification.	Fulfills most requirements; solves the issue without manual edits	Flawless "Zero-Shot" execution; robust handling of all edge cases.
Efficiency	Poor runtime logic; high memory leaks or usage; unneeded complexity	Uses "naive" approaches; reasonable time but excessive resources.	Appropriate algorithms/libraries used; runs in reasonable time.	Optimized runtime complexity; minimal memory footprint; Not Complex
Readability	Disorganized; no comments; inconsistent naming; poor formatting.	Basic organization but vague naming; "Magic Numbers" present.	Well-organized; meaningful naming; consistent indentation/docstrings.	Self-documenting code; high cognitive clarity; follows all style guides
Maintainability	Monolithic; hardcoded; adding features breaks existing systems.	Rigid structure; some modularity but high coupling between parts.	Modular design; uses functions/classes; avoids hardcoding.	Highly extensible; decouples logic; "Open-Closed" principle applied

Figure 5: Evaluating Generated Code by Scale

The rubric for game playability is taken from Mike Compton's “Game Evaluation Criteria in 5 minutes or Less” as it provides the basic categories that constitute key sections of making a video game playable. "The use of standardized evaluation rubrics is essential for mitigating subjectivity in game selection.” As noted by Gunter et al. (2008), formal design paradigms are required to align gameplay mechanics with learning outcomes. In practice, frameworks such Compton's “Video Game Evaluation Rubric” have become foundational in the field, providing a structured 1–7 scale for variables like Integration and Flow, which are recognized by the American

Library Association as critical benchmarks for collection development (Johnson, 2018)." One of Compton's major points when designing this criteria is that it can be used to “effectively evaluate a prototype - regardless of what stage of development it is in,” allowing us to use this rubric on each stage of the experiment (Compton 2014). Compton also includes a section about “Category Biases” where the person reviewing the game should self-report their own biases against a specific genre or category such as FPS or games with a strong narrative. For sake of reproducibility, we will focus specifically on the criteria given below in figure x when evaluating these games. Compton also includes a section for feedback based on: Strongest Point, Weakest Point, One Change, and Comparable Games. Instead of separating these out we will combine them into each section as defined below such that when asked about a games Clarity we will also showcase the pros and cons of that game within this category. The provided criteria has a scale of 1-7 such that one represents poor, being broken, cumbersome, or completely fails in this category. Four is average, a functional but flawed game. It has "somewhat" clear rules or "mostly" okay mechanics but lacks polish. Seven is excellent or a streamlined, fair, and deeply engaging game with no unnecessary procedures. The evaluation criteria has 6 defined sections being Clarity, Flow, Balance, Length, Integration, and Fun. Definitions of these sections can be seen below in figure 6.

	1 → 2	→ 3	→ 4	→ 5 → 6 → 7
Clarity	Very cumbersome board design / Hard for the players to see what is going on in the game / Rules are very unclear and difficult to understand.	Somewhat cumbersome layout / Rules are somewhat unclear and difficult to understand.	Somewhat streamlined layout / Rules are somewhat clear and easy to understand.	Very streamlined layout / The players can easily see what is going on in all areas of the game / Rules are very clear and easy to understand (no areas of ambiguity).
Flow	Lots of unnecessary procedures / Lots of fiddliness / Lots of cumbersome exceptions to rules / Needs lots of streamlining.	Several unnecessary procedures / Several areas of fiddliness / Several cumbersome exceptions to rules.	Few unnecessary procedures / Few areas of fiddliness / Few exceptions to rules / Mostly streamlined but with a few exceptions.	No unnecessary procedures / Minimal to no fiddliness / Very few if any exceptions to rules / Very streamlined.
Balance	Very imbalanced / Broken / Run-away leader problem / The impact of luck is much too significant for the game to be fair.	More imbalanced than balanced / A few areas work but many have strategic loopholes / Many of the luck elements are too significant but a few are appropriate.	More balanced than imbalanced / Has some small areas with loopholes that need fixing / Many of the luck elements are appropriate but a few are still too significant.	Very balanced game that is fair for all players / No strategic loopholes / Any and all elements of luck in the game are appropriate in their significance.
Length	The game is extremely too short or too long for what it offers.	The game is mostly too short or too long for what it offers.	The game's length is somewhat too short or too long for what it offers.	The game's length is appropriate for what it offers.
Integration	The mechanics and the theme are extremely mismatched / The different mechanical elements of the game do not compliment each other or "fit" together at all.	The mechanics and the theme are somewhat mismatched / Several of the mechanics do not "fit" into the greater whole of the game.	The mechanics and the theme are a fairly good match / A few elements of the game do not "fit" well but most of them do.	The mechanics and the theme strongly compliment each other / The various elements strongly integrate to create a unified game.
Fun	Complete lack of emotional connection or tension throughout the game / Lots of "downtime" / No interesting decisions offered / Very uninteresting theme / The game was not fun to play at all.	Very few moments of emotional connection or tension during the game / Somewhat uninteresting theme / The moments of fun were rare occurrences.	Some moments of emotional connection or tension during the game / Somewhat interesting theme / The game was somewhat fun to play but still had a few areas that were lacking.	Consistent emotional connection or tension throughout the game / Very interesting theme that strongly engages the imagination of the players / The game was very fun to play.

Figure 6: Compton's Game Evaluation Criteria (5 Minutes or Less)

Case Studies of AI-Generated Games

The following games were generated by two distinct iterations of ChatGPT: GPT 4.3 and GPT 5.5. Each model was tested using the discussed prompting methods Zero-shot, Single/Few-shot, and Simulated Tuning. The finished product for each prompting method was subsequently assessed against the established four-pillar Code Quality Rubric (Functionality, Efficiency, Readability, and Maintainability) and Compton's Game Playability Rubric. The raw generated code and corresponding chat logs for these case studies can be found in the Project_Repository. All prompting strategies were given the goal of making a game about grape picking.

AI Game 1 & 2: Grape Picking Game- Zero shot prompt

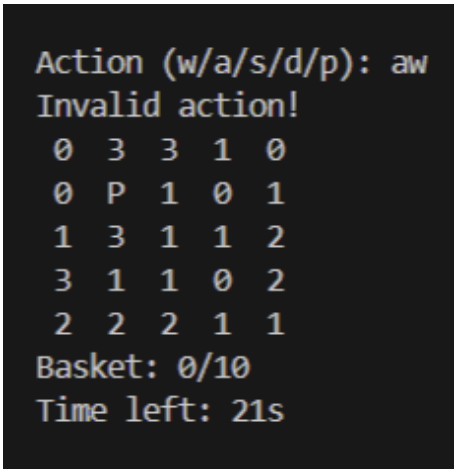

Figure 7: GPT 4.3_Zero – AI Game 1

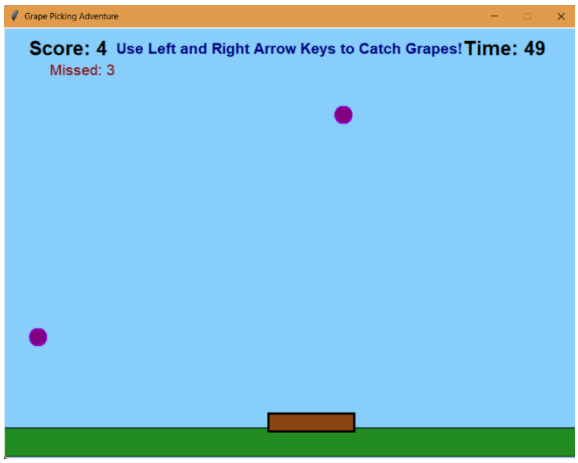

Figure 8: GPT 5.5_Zero – AI Game 2

Code Quality Evaluation: GPT 4.3 (Zero-Shot)

The game produced by GPT 4.3 operates as a terminal based grid game where the player must navigate a 5x5 area to pick grapes within a time limit.

Functionality (Score 3): The model successfully produced an executable, on-topic game that required no manual modifications to run. It effectively implements grid boundaries and input validation, accepting both upper and lower-case commands. However, it falls short of an exemplary score due to a critical logical flaw in its time-tracking mechanism. The game loop relies on the standard input() function, which halts execution and waits indefinitely for the user (Figure 9). Consequently, if the timer expires while waiting for user input, the game state does not update in real-time, failing to trigger the "Game Over" state until a subsequent keystroke is made.

```
while time.time() - start_time < TIME_LIMIT and basket < BASKET_CAPACITY:
    print_vineyard()
    action = input("Action (w/a/s/d/p): ").lower()
    if action in ["w", "a", "s", "d"]:
        move_player(action)
    elif action == "p":
        pick_grapes()
    else:
        print("Invalid action!")
```

Figure 9: Code Excerpt from 4.3_Zero, Time Tracking Flaw

Efficiency (Score 2): The programmatic efficiency is heavily compromised by the timer bug, which allows the script to continue consuming resources indefinitely. While the use of the random and time libraries successfully abstracts necessary game logic, the spatial complexity is suboptimal. The game utilizes a nested list structure for the 5x5 grid, resulting in a space complexity of $O(N^2)$. The grid is largely sparse, populated mostly by zeroes, implementing a sparse matrix would have reduced the space complexity to O(G), where G represents the number of collectible grapes (Figure 10). Time complexity remains acceptable, utilizing O(1) direct indexing for grid lookups.

```
vineyard = [[random.randint(0, 3) for _ in range(VINEYARD_SIZE)]for _ in range(VINEYARD_SIZE)]
```

Figure 10: Code Excerpt from 4.3_Zero Space Time Complexity

Readability (Score 3): The codebase demonstrates competent communicative quality. Variables are descriptively named, and functional abstraction is used appropriately to handle movement and terminal rendering. It loses points primarily due to the inconsistent application of documentation, while some loops are explained, several core functions lack necessary docstrings or inline comments (Figure 11). Additionally, the code contains "magic numbers" related to list indexing.

```
def print_vineyard():
    for i in range(VINEYARD_SIZE):
        row = ""
        for j in range(VINEYARD_SIZE):
            if player_pos == [i, j]:
                row += " P "
            else:
                row += f" {vineyard[i][j]} "
        print(row)
```

Figure 11: Code Excerpt from 4.3_Zero, Lack of Comments

Maintainability (Score 2): The structural integrity of the code is rigid. The primary game loop is implemented as a monolithic block, whereas initialization and termination states should have been abstracted into separate functions. The reliance on global variables to manage state further degrades maintainability, creating a tightly coupled system that would require a complete rewrite to accommodate scalable features, such as larger grid sizes or multiplayer support.

Code Quality Evaluation: GPT 5.5 (Zero-Shot)

The output from GPT 5.5 interpreted the prompt differently, resulting in an arcade-style game focused on catching falling grapes rather than picking them from a grid.

Functionality (Score 4): The script executes flawlessly out-of-the-box. The game logic successfully restricts player movement to the screen boundaries, accurately tracks both collected and missed grapes, and dynamically updates the game timer. There are no identifiable syntax or runtime errors, representing a robust zero-shot execution.

Efficiency (Score 3): The execution utilizes appropriate resource management techniques. The model successfully caps the frame rate at 50 FPS to normalize runtime speed and properly implements deletion methods to prevent memory leaks. However, it duplicates list resources for update checks, the program utilizes shallow copies rather than referencing the original data structures (Figure 12).

```
for grape in self.grapes[:]:
```

Figure 12: Code Excerpt from 5.5_Zero, Shallow Copies

Readability (Score 1): The communicative quality of the code is unsatisfactory. The script contains zero comments or docstrings (Figure 13), making the underlying logic difficult to parse at a glance. While it uses meaningful variable names and applies object-oriented principles to

abstract behaviors into classes, the complete lack of documentation severely increases the cognitive load required for peer review.

```
class Grape:
    def __init__(self, canvas):
        self.canvas = canvas
        self.x = random.randint(20, WIDTH - 20)
        self.y = -20
        self.speed = random.randint(START_SPEED, START_SPEED + 3)
        self.id = canvas.create_oval(
            self.x,
            self.y,
            self.x + GRAPE_SIZE,
            self.y + GRAPE_SIZE,
            fill="purple",
            outline="dark violet",
            width=2,
        )
```

Figure 13: Code Excerpt from 5.5_Zero, Magic Numbers with Lack of Documentation

Maintainability (Score 2): While the codebase makes strides toward modularity by utilizing independent classes, it suffers from a heavy reliance on global variables. Furthermore, the main game loop remains overloaded with procedural tasks, creating a fragile architecture where the addition of new features is highly likely to break existing systems.

Game Playability Evaluation

The interactive quality of both games was assessed using the 1–7 scale defined by Compton's Game Evaluation Criteria. The results are compared in the table below (Figure 14).

Metric	GPT 4.3 Evaluation	Score	GPT 5.5 Evaluation	Score
Clarity	Uses a cumbersome terminal interface but clearly defines controls upfront.	**3**	Features a highly streamlined, visually intuitive layout, but fails to explicitly provide player controls.	**5**
Flow	Hindered by unnecessary procedures, requiring a dedicated keystroke to "pick up" items rather than relying on positional collision.	**3**	Highly streamlined with no unnecessary inputs; movement inherently triggers collection.	**7**
Balance	Relies entirely on RNG for board generation, resulting in inconsistent difficulty and unfair time constraints.	**3**	Relies on RNG for standard and golden grape spawns, creating inconsistent pacing without guaranteed fairness.	**3**
Length	The terminal gameplay loop feels excessively long and tedious for the limited mechanical depth it offers.	**2**	The 60-second timer feels highly appropriate, maintaining engagement throughout the entire session.	**7**
Integration	The mechanics accurately represent the literal action of "picking" grapes from a field.	**6**	Interprets the theme as "catching" rather than "picking," but integrates this mechanic flawlessly into the overall visual theme.	**5**
Fun	Lacks emotional tension, strategic decision-making, or general engagement.	**1**	Provides consistent tension and engagement through active gameplay and the inclusion of high-value "golden grapes."	**7**

Figure 14: Gameplay Evaluation of Zero-Shot Prompting Under Compton's Rubric

Summary of Results

Despite shifting the mechanical interpretation of the prompt from "picking" to "catching," GPT 5.5 produced a significantly more engaging and playable product, scoring nearly double the playability average of GPT 4.3. However, the technical evaluation reveals that both models remain tied in underlying code quality. While GPT 5.5 provided a more robust out-of-the-box execution, both models exhibited notable deficiencies in standard documentation practices and long-term codebase maintainability.

Average Code Quality Score (GPT 4.3): 2.50 / 4.0

Average Code Quality Score (GPT 5.5): 2.50 / 4.0

Average Playability Score (GPT 4.3): 3.00 / 7.0

Average Playability Score (GPT 5.5): 5.67 / 7.0

AI Game 3 & 4: Grape Picking Game - Single/Few Shot Prompt

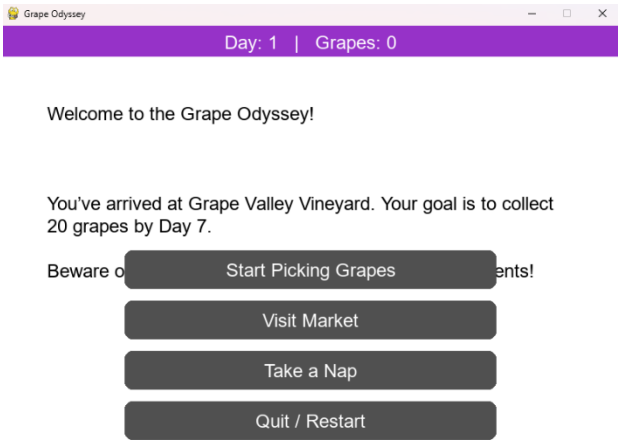

Figure 15: GPT 4.3_Single – AI Game 3

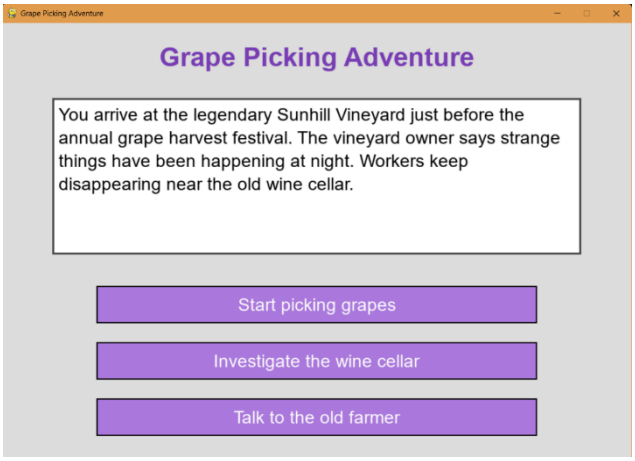

Figure 16: GPT 5.5_Single – AI Game 4

Code Quality Evaluation: GPT 4.3 (Single/Few-Shot)

The output from GPT 4.3 resulted in a highly flawed graphical application that struggled to handle state management and basic rendering efficiently.

Functionality (Score 2): The generated code is marginally functional but suffers from significant bugs and formatting issues that severely impact the user experience. Key issues include text rendering behind interactive buttons, UI elements being cut off from the screen, and the initial omission of a requested grape counter. Furthermore, the game logic fails to restrict player actions after the intended end-state (Day 7), allowing the day counter to increase indefinitely. Finally, the "Restart" feature fails to function, and attempting to force the model to add previously discussed features ultimately resulted in broken code.

Efficiency (Score 1): The programmatic architecture is highly unoptimized and demonstrates a fundamental misunderstanding of game loop processing. The code continuously performs heavy, redundant string parsing, list generation, and text rendering on every single frame. Instead of allocating text surfaces and button rectangles once upon a state change, the program completely recreates these objects every 33 milliseconds (30 frames per second) (Figure 17). Most egregiously, the narrative logic relies on parsing gameplay variables directly out of display strings using the split() function (Figure 18). This forces the Python interpreter to create temporary list objects and perform regex-like operations continuously. As a fundamental principle of software design, the underlying data state should drive the text display, not the reverse. The only minor saving grace is that button objects are successfully garbage-collected during state transitions.

```
for line in lines:
    if y > max_height:
        break  # prevent overlapping buttons
    rendered = font.render(line.strip(), True, BLACK)
    surface.blit(rendered, (margin, y))
    y += FONT_SIZE + 5
```

Figure 17: Code Excerpt from 4.3_Single, Rendering Allocation

```
if "+" in event:
    added = int(event.split("+")[1].split()[0])
    self.grapes += added
elif "Lose" in event:
    lost = int(event.split()[2])
    self.grapes = max(0, self.grapes - lost)
```

Figure 18: Code Excerpt from 4.3_Single, String Gameplay Logic

Readability (Score 1): The communicative quality of the code is unsatisfactory. While comments exist to visually separate functions, they fail to explain the underlying logic or purpose of those functions. The codebase relies heavily on "magic numbers," and despite some attempts at abstraction, the functions themselves remain massive and difficult to parse.

Maintainability (Score 2): The structure is brittle. The reliance on runtime string parsing (split()) to determine logic branches creates a highly fragile system. The presence of large, monolithic functions and magic numbers severely limits future development. While there is a baseline level of high-level modularity, the architectural foundation is too unstable to support meaningful feature additions without refactoring.

Code Quality Evaluation: GPT 5.5 (Single/Few-Shot)

GPT 5.5 produced a significantly more stable and modular text-based adventure, successfully utilizing multiple files to organize the codebase.

Functionality (Score 4): The game executes flawlessly out-of-the-box. It successfully delivers a multi-path grape-picking adventure with distinct endings. The user interface is properly formatted, and there are no noticeable runtime errors, logical bugs, or visual overlapping issues.

Efficiency (Score 2): While the model efficiently stores the narrative nodes within a dictionary structure for O(1) fast lookup, it inherits the same fundamental rendering flaws seen in GPT 4.3.

The game loop continuously generates new button rectangles and re-renders font surfaces 60 times a second, failing to cache static UI elements between state changes.

Readability (Score 1): Despite superior architectural organization, the code entirely lacks docstrings and inline comments. While it utilizes well-defined variable names and separates gameplay logic into distinct program files, the complete absence of documentation and the lingering presence of some magic numbers results in a high cognitive load for developers attempting to review the code.

Maintainability (Score 3): The model avoids monolithic structures by dividing major functional components across multiple distinct Python files. The narrative is stored in a large story nodes dictionary, which is an appropriate and modular data structure for this genre. This overall design makes the codebase highly adaptable for future updates, though the scattered magic numbers prevent a perfect score (Figure 19).

```
start_y = 420
    for index, choice in enumerate(node["choices"]):
        button = Button(
            150,
            start_y + index * 90,
            700,
            60,
            choice[0],
            FONT,
            LIGHT_PURPLE,
            PURPLE,
            WHITE
        )
```

Figure 19: Code Excerpt from 5.5_Single, Magic Numbers Present

Game Playability Evaluation

The interactive quality of both games was assessed using the 1-7 scale defined by Compton's Game Evaluation Criteria. The results are compared in the table below (Figure 20).

Metric	GPT 4.3 Evaluation	Score	GPT 5.5 Evaluation	Score
Clarity	The core "click adventure" mechanics are inherently easy to understand, but early instructions are obscured due to text rendering behind UI buttons.	**5**	The text fits perfectly within the designated UI boxes, making the layout and objective immediately clear to the user.	**7**
Flow	Features streamlined interactions, but the fundamental failure of the "Restart" button breaks the gameplay loop upon completion.	**5**	Highly streamlined. The restart functionality works perfectly, allowing for a seamless cyclical gameplay loop.	**7**
Balance	The narrative choices and outcomes feel fair and balanced.	**7**	The narrative choices, branches, and outcomes feel fair and balanced.	**7**
Length	The overall content is somewhat sparse; the limited number of narrative paths makes the experience feel slightly too short.	**5**	Similar to 4.3, the adventure lacks deep branching paths, resulting in an experience that feels slightly too brief.	**5**
Integration	Effectively integrates a functional "grape counter" metric, marrying the narrative choices to a tangible mechanical goal.	**6**	The text successfully conveys the theme, but it lacks the dedicated tracking mechanics (like a visible counter)	**5**
Fun	Hindered by the broken restart loop and a lack of narrative variety, limiting replay ability and engagement.	**4**	Provides a highly engaging experience with a stronger narrative, tangible tension (e.g., the "grape beast"), and a functional replay loop.	**7**

Figure 20: Gameplay Evaluation of Single/Few-Shot Prompting Under Compton's Rubric

Summary of Results

In the guided development scenario, GPT 5.5 vastly outperformed GPT 4.3 in both codebase architecture and user experience. GPT 4.3 struggled under the weight of iterative prompting, resulting in broken features, UI overlapping, and highly brittle string-parsing logic. Conversely, GPT 5.5 successfully leveraged modular, multi-file architecture to produce a stable and highly playable narrative game. However, both models demonstrated a shared, fundamental misunderstanding of GUI rendering efficiency by continuously redrawing static assets frame-by-frame, and both entirely neglected standard code documentation practices.

Average Code Quality Score (GPT 4.3): 1.50 / 4.0

Average Code Quality Score (GPT 5.5): 2.50 / 4.0

Average Playability Score (GPT 4.3): 5.33 / 7.0

Average Playability Score (GPT 5.5): 6.33 / 7.0

AI Game 5 & 6: Grape Picking Game - Simulated Tuning & SCRUM

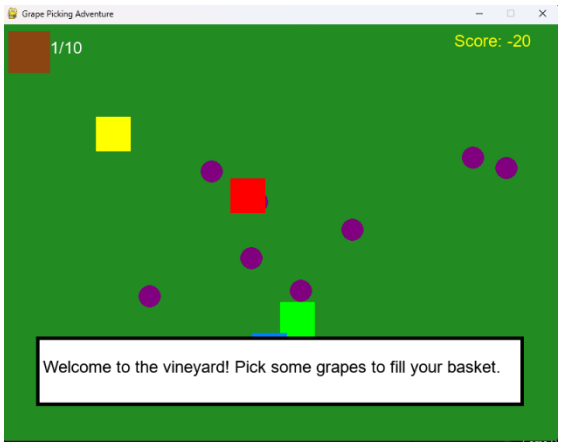

Figure 21: GPT 4.3_Sim – AI Game 5

Figure 22: GPT 5.5_Sim – AI Game 6

Code Quality Evaluation: GPT 5.5 (Simulated Tuning)

Despite the simulated iterative prompting, GPT 5.5 struggled significantly with initial compilation, producing an unoptimized and tightly coupled architecture.

Functionality (Score 1): The generated code failed to run out-of-the-box. As documented in the execution logs, the script contained significant syntax and runtime errors that prevented compilation. The code required considerable manual debugging and human intervention to reach an executable state, placing it firmly in the "Unsatisfactory" tier of the rubric. Once manually fixed, it successfully delivered a game about grape picking.

Efficiency (Score 2): The graphical rendering and entity management systems are highly unoptimized. The main game loop (run()) executes without frame-rate throttling, running as fast as the host processor allows and wastefully processing graphics on every single frame. Furthermore, spatial lookups for non-playable characters (NPCs) utilize an O(n) linear search rather than an optimized O(1) spatial hash or dictionary lookup (Figure 23).

```
def maria_event(player, npcs):
    maria = None
    for npc in npcs:
        if npc.name == "Maria":
            maria = npc
```

Figure 23: Code Excerpt from 5.5_Sim, NPC Linear Search

Readability (Score 1): The communicative quality is severely lacking. The codebase is entirely devoid of comments and docstrings. While it utilizes meaningful variable names and separates logic into functions, the complete lack of documentation combined with the presence of "magic numbers" creates a high cognitive barrier for code review.

Maintainability (Score 2): The model attempts a modular architecture by utilizing classes and functions, but the execution is flawed. The functions exhibit tight coupling, relying heavily on the internal, private details of other classes. Additionally, coordinate positioning formulas are hardcoded into the logic rather than being exposed as configurable constants, making future UI adjustments highly rigid.

Code Quality Evaluation: GPT 4.3 (Simulated Tuning)

GPT 4.3 successfully produced an executable game, but the underlying logic was riddled with game-breaking bugs and structural "spaghetti code."

Functionality (Score 1): While the game technically runs out-of-the-box without compilation errors, the gameplay logic is fundamentally broken. Critical bugs include the player character getting permanently stuck on NPCs, the ability to walk entirely out of the screen boundaries, and a score counter that erroneously drops into negative integers. Furthermore, the game loop lacks a win state, fail state, or restart function; fulfilling the grape quota does not trigger an end-game sequence.

Efficiency (Score 2): The model demonstrates an uneven understanding of system resources. Positively, it implements proper framerate throttling and successfully pre-renders vector shapes outside of the main loop. However, it critically fails to pre-render text surfaces, forcing the engine to execute costly font-rendering algorithms on every single frame. Additionally, AI pathfinding relies on an O(n) linear search. Scaling this game to hundreds of active entities and evaluating that collection linearly on every loop iteration would cause severe framework stutters (Figure 24).

```
target_grape = next((g for g in grapes if g[2]), None)
```

Figure 24: Code Excerpt from 4.3_Sim, Linear Search

Readability (Score 2): The code contains a baseline level of structural readability. Meaningful variable names are utilized, and a decent volume of comments successfully separates major sections of the code. However, the absence of docstrings and the presence of a massive, monolithic main game loop heavily congested with nested if blocks significantly degrades overall legibility.

Maintainability (Score 1): The architectural integrity is highly unsatisfactory. The entire game is contained within a single-file, monolithic structure that is highly reliant on global variables and state mutations. Furthermore, entity tracking (such as grape locations) is managed using raw index positions within a multidimensional list (Figure 25), which is an inherently fragile and poorly scaling data structure. This lack of abstraction creates a brittle codebase that strongly resists feature expansion.

```
# --- Grapes ---
num_grapes = 15
grapes = [[random.randint(150, WIDTH-50), random.randint(150, HEIGHT-150),
True] for _ in range(num_grapes)]
```

Figure 25: Code Excerpt from 4.3_Sim, Multidimensional Locations

Game Playability Evaluation

The interactive quality of both games was assessed using the 1-7 scale defined by Compton's Game Evaluation Criteria. The results are compared in the table below (Figure 26).

Metric	GPT 4.3 Evaluation	Score	GPT 5.5 Evaluation	Score
Clarity	The game completely lacks on-screen instructions. The core rules are highly ambiguous, and the movement controls are not immediately intuitive to the player.	1	Fails to clearly define the game's core objectives or rules. While the primary display is straightforward, visual overlapping obscures critical stats, and the feedback text fails to explain *why* certain variables change.	2
Flow	Features streamlined arrow-key movement without unnecessary procedures. However, the lack of a "Restart" button and the high probability of entering an endless, unwinnable loop creates severe mechanical friction.	5	Highly streamlined, limiting player choices to three clearly defined action buttons per day. However, it also suffers from the omission of a functional "Restart" loop.	6
Balance	Fundamentally unbalanced. The NPC logic is programmed with extreme aggression, allowing them to collect all resources before the player. In roughly 90% of playthroughs, this results in an inescapable endless loop.	1	Lacks fundamental balancing deterrents. A player can effortlessly exploit the game by repeatedly selecting the same action, as secondary narrative stats (e.g., health, NPC trust) have zero impact on the final outcome.	1
Length	The structural flaw that traps the player in an endless loop renders the gameplay duration highly erratic, fluctuating between appropriately short or infinitely long.	1	The game’s designated length is highly appropriate for the limited mechanical depth it offers, providing a concise experience.	7

Integration	The mechanical actions (physically moving a character to collect grapes and outpace rival NPCs) strongly complement the core theme of a competitive agricultural harvest.	7	The mechanics and theme are severely mismatched. The focus shifts entirely away from the physical act of "picking grapes" to an abstract resource-management game about surviving a harvest week by passing unseen statistical thresholds.	1
Fun	The aggressive NPC AI generates genuine tension and urgency; however, this is severely undermined by collision bugs that permanently trap the player upon contact, ruining the experience.	5	The game attempts to foster emotional connection through narrative NPCs, but fails completely because the player's choices and secondary stats have no tangible effect on the game's outcome.	1

Figure 26: Gameplay Evaluation of Simulated Tuning Under Compton's Rubric

Summary of Results

The Simulated Tuning approach yielded the poorest overall results across the case studies. Despite the iterative feedback loop, both models struggled to produce structurally sound or mechanically engaging games. GPT 4.3 generated an unbalanced, highly buggy graphical game that frequently trapped players in unwinnable endless loops, completely undermining its strong thematic integration. Conversely, GPT 5.5 produced a highly stable but functionally hollow resource-management game, where the underlying mechanics were disconnected from both the established theme and the player's choices. Ultimately, both models tied for the lowest Code Quality scores in the study, heavily weighed down by a shared inability to optimize basic rendering systems or apply foundational documentation practices.

Average Code Quality Score (GPT 4.3): 1.50 / 4.0

Average Code Quality Score (GPT 5.5): 1.50 / 4.0

Average Playability Score (GPT 4.3): 3.33 / 7.0

Average Playability Score (GPT 5.5): 3.00 / 7.0

AI Game 7: Human-AI Collaborative Development

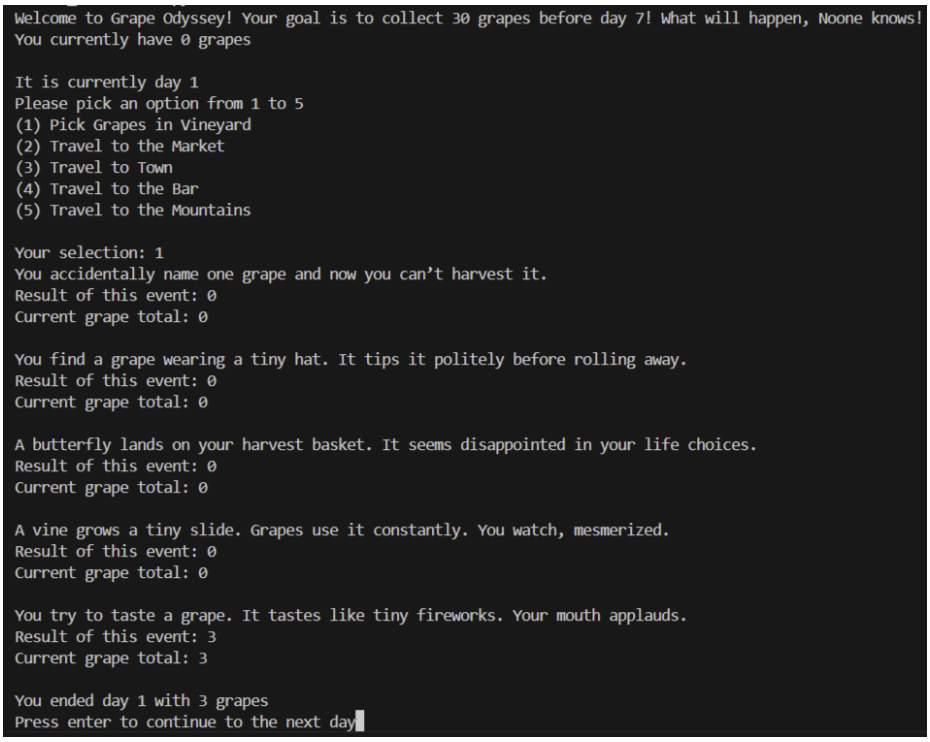

Figure 27: Human-AI Collab – AI Game 7

Code Quality Evaluation: AI Game 7 (Human-AI Collaborative Development)

This section examines the fourth case study, which evaluates a game generated via a Human-AI collaborative development approach. Under this methodology, the human designer and the AI worked iteratively to produce a text-based strategy and adventure game. The core gameplay loop requires the player to meet a specific grape harvest quota by the end of a seven-day cycle, offering multiple tactical paths and narrative choices to achieve this objective.

Functionality (Score: 4): The collaborative development codebase achieves flawless execution, operating successfully out-of-the-box without requiring any post-generation manual syntax corrections or runtime interventions. The program comprehensively fulfills all design constraints, accurately tracking the seven-day timeline, enforcing the grape harvest quota, and smoothly managing state transitions across varied player choices. No runtime edge cases or system-breaking bugs were observed during execution.

Efficiency (Score: 3): The codebase demonstrates strong systems-level optimization balanced by a minor localized bottleneck. Uses a dispatch dictionary, that functions as a v-table, (Figure 28) for instantaneous, O(1) constant-time function lookups, completely avoiding the overhead of deep conditional branches. Pre-compiles key datasets at the module level to eliminate redundant memory allocations and utilizes immutable named tuples to secure game states against accidental overwrites. The Mountain Events function relies on standard if/elif/else blocks, forcing sequential evaluation that degrades performance linearly ,O(n) time complexity, for conditions at the bottom of the tree.

```
ACTIONS = {
    1: events.VineyardEvents,
    2: events.MarketEvents,
    3: events.TownEvents,
    4: events.BarEvents,
    5: events.MountainEvents
}
```

Figure 28: Code Excerpt from Main in AI_Collab, Dispatch Dictionary

Readability (Score: 3): The communicative quality of the code is highly robust. The codebase features comprehensive inline comments, structural section dividers, and detailed docstrings that explicitly articulate the underlying engineering rationale behind specific implementations (Figure 29). Variables are predominantly bound to meaningful naming conventions; however, some are

not. While the script contains several literal values ("magic numbers"), their presence is mitigated by accompanying explanatory comments that clarify their operational context.

```
# Establish casino weights: 40% win, 50% lose, 10% jackpot
# WHY: random.choices allows us to skew probabilities easily
#without complex math.
outcome = random.choices(["win", "lose", "jackpot"],
weights=[40,50,10], k=1)
```

Figure 29: Code Excerpt from Main in AI_Collab, Extensive Documentation

Maintainability (Score: 3): The high-level structural design is highly extensible. Utilizing an autonomous Options class and an integrated Actions mapping array, the game satisfies the "Open-Closed" principle, allowing developers to introduce top-level structural expansions or new menu options with minimal friction. However, architecture falls short of an exemplary score due to the monolithic design of the Mountain Events function. Unlike the modular, data-driven framework seen in Vineyard Events, the mountain narrative branches remain tightly coupled within hardcoded execution blocks rather than being fully decoupled into an external data configuration.

Game Playability Evaluation

The interactive quality of the collaboratively developed game was assessed using the 1-7 scale defined by Compton's Game Evaluation Criteria. The results are compared in the table below (Figure 30).

Metric	Human-AI Collaborative Game Evaluation	Score
Clarity	The game establishes explicit narrative goals and parameters at initialization, including a clear 30-grape harvest target. It could more explicitly prompt the player regarding required keyboard inputs, though user interaction remains intuitive.	**6**
Flow	The user interface presents clean, well-defined menu choices that cleanly guide the player through daily strategic options without introducing jarring mechanical friction.	**7**
Balance	Probability matrices and random-number generation (RNG) are balanced fairly across all risk-reward mechanics. The systems are designed symmetrically, preventing the user from exploiting procedural loopholes.	**7**
Length	The pacing is exceptionally balanced for a text-based strategy game; the seven-day framework gives players a complete, satisfying narrative arc within a single session.	**7**
Integration	Mechanics are tied directly to the grape-harvesting motif, requiring players to balance agricultural management with riskier expeditions. The thematic integration is strong, though it could be further enhanced by incorporating a dedicated real-time mini-game.	**5**
Fun	The inclusion of branching high-stakes choices, ranging from high-risk exploration vectors to humorous random events, creates high emotional tension and excellent engagement.	**7**

Figure 30: Gameplay Evaluation of Human-AI Collab Under Compton's Rubric

Summary of Results

The Human-AI collaborative development approach yielded a highly polished, robust, and mechanically engaging end product. By combining human structural oversight with AI generation, the codebase avoids the performance flaws and brittle logic seen in pure zero-shot iterations, delivering optimized O(1) dispatch architectures and secure immutable data structures. While

localized areas of the narrative logic remain structurally rigid, the game achieves the highest cumulative playability and structural quality marks among the evaluated models.

Average Code Quality Score: 3.25 / 4.0

Average Playability Score: 6.50 / 7.0

Findings and Analysis

An additional layer of analysis to remove any potential observational bias in the evaluation section was to blindly survey 10 users to rank the games produced in the project. Blindly refers to not telling the users that the games were created by AI nor that game 7 was a joint Human-AI collaboration game. A survey was created that showcases gameplay of each game and asks the user to rank each game based on the same Compton's game evaluation rubric.

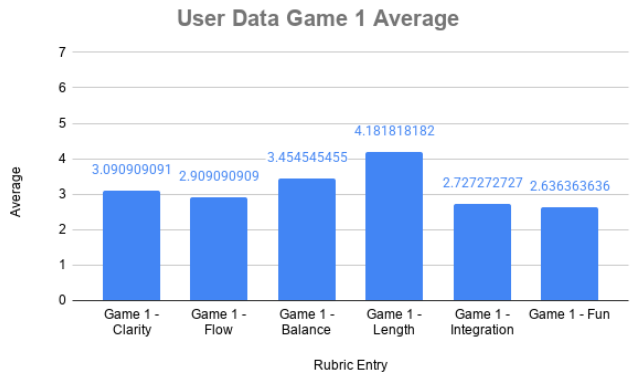

Figure 31: Game 1 Average by Rubric Item

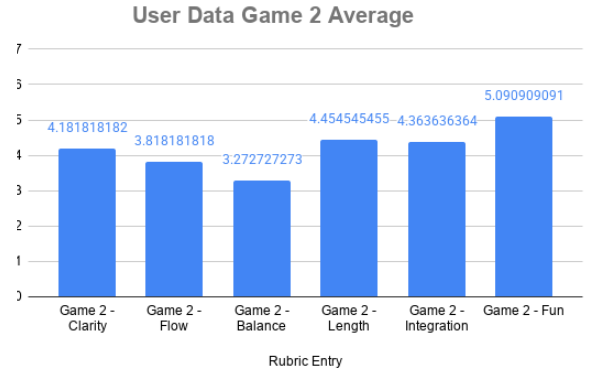

Figure 32: Game 2 Average by Rubric Item

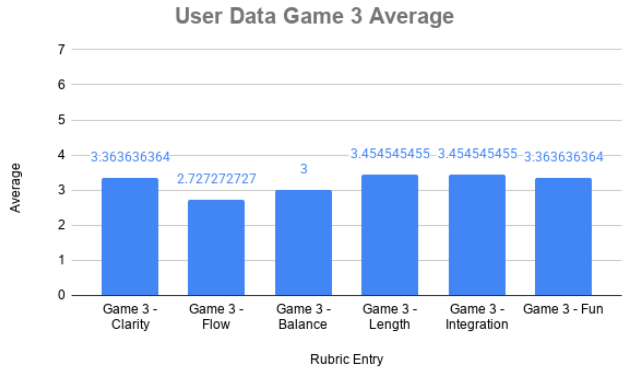

Figure 33: Game 3 Average by Rubric Item

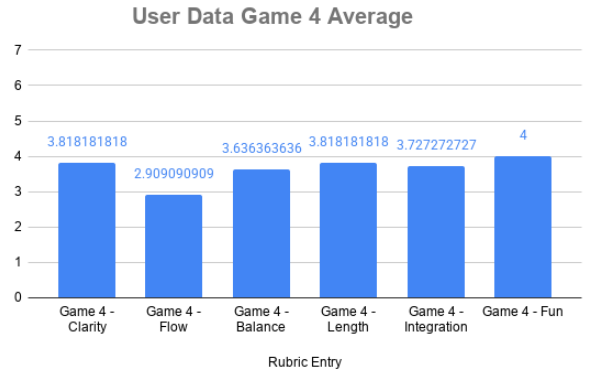

Figure 34: Game 4 Average by Rubric Item

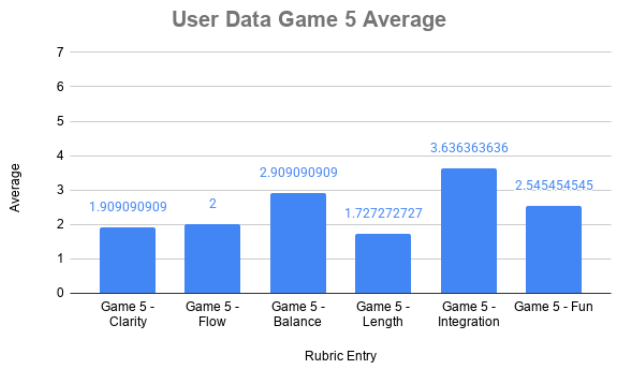

Figure 35: Game 5 Average by Rubric Item

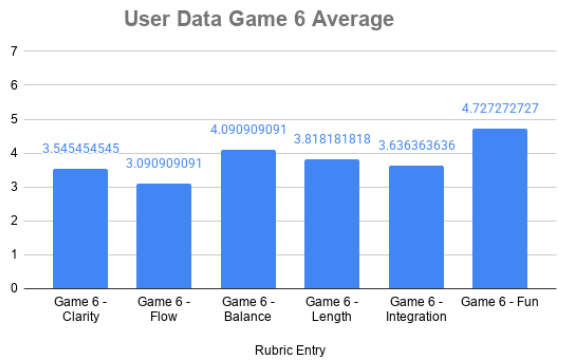

Figure 36: Game 6 Average by Rubric Item

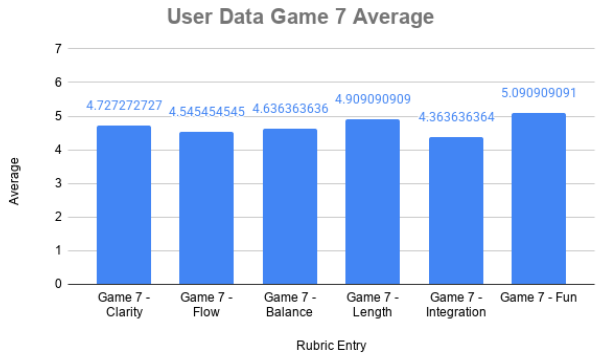

Figure 37: Game 7 Average by Rubric Item

Method	Model	Code Quality	Playability
Zero-Shot	4.3	2.5/4	3/7
	5.5	2.5/4	5.67/7
Single/Few-Shot	4.3	1.5/4	5.33/7
	5.5	2.5/4	6.33/7
Simulated Tuning	4.3	1.5/4	3.33/7
	5.5	1.5/4	3/7
Human-AI Collab	Hybrid	3.25/4	6.5/7

Figure 38: Summary of Average Scores during Evaluation

A review of the Code Quality metrics reveals a counterintuitive trend, such that as prompting complexity and iterative guidance increased, the structural integrity of the generated code generally degraded. GPT 4.3 achieved 2.5 in the Zero-Shot trial but dropped to a 1.5 during both Few-Shot and Simulated Tuning. Similarly, GPT 5.5 maintained a 2.5 through Few-Shot but ultimately fell to 1.5 during Simulated Tuning. In a Zero-Shot environment, the model is "provided maximum convenience, potential for robustness, and avoidance of spurious correlations," the model is afforded maximum creative liberty with minimal constraints (Brown 2020). As the methodology shifted toward Simulated Tuning, which requires the model to adhere to specific design constraints and iterate upon prior outputs, the system struggled under the weight of accumulated constraints. As the chat goes on, "Semantic similarity between the target information and the query decreases, performance degradation accelerates with increasing input length" (Chromadb 2025). Similarly, "Model performance drops by an average of 39% across sequential interactions, accompanied by a 112% increase in output unreliability" (Laban 2025). This degradation can be attributed to three primary phenomena: Context Drift, Hallucination, and Token-Level Optimization.

Context drift is "the gradual divergence of a model's outputs from goal-consistent behavior across turns," or the devolving of output as the model runs out of its limited storage to preserve the initials chats context (Dongre 2025). In the experiment this was seen in the Few-shot and Simulated Tuning games, such that as more prompts were given to the models, the code became worse and was prone to more errors over time in relation to the Zero-shot prompt. Context drift tends to lead to hallucinations, as conversational length taxes the context window, models display a severe vulnerability to hallucination. In an architectural context, hallucination is defined as the phenomenon where an AI model generates text that is "highly fluent, syntactically correct, and

sounds plausible, but is factually incorrect, nonsensical, or entirely unfaithful to the provided source input" (Ziwei 2023). Hallucination can be separated into Intrinsic and Extrinsic Hallucination. Intrinsic Hallucination refers to when the model's output "contradicts the source material or constraints provided by the user in the prompt" (Ziwei 2023). Classic examples include when the model provides wrong dates for a search query, or in our case attempts to make a game that isn't necessarily about grape picking, like game 2. This was observed when the model repeatedly omitted requested features, such as core game loops, or flatly failed to implement a weighted probability array within a random calculation loop despite explicit scripting instructions (Figure 39). Extrinsic Hallucination refers to the model generating new information that "cant be supported or verified by established knowledge", essentially fabricating data out of thin air (Ziwei 2023). In the study this can be seen when the model claims to have fixed a bug, or claim understanding of what was causing an error without fixing it (Figure 39).

```
outcome = random.choices(["win", "lose", "jackpot"], weights=, k=1)
# answer is just
outcome = random.choices(["win", "lose", "jackpot"], weights=[40,50,10], k=
```

Figure 39: AI Fails to Implement Simple Change

The difference between the pure AI-generated code and the Human-AI Collaborative Game exposes a fundamental mismatch in software construction strategies. Human engineers approach project initialization with "systems-level thinking," mapping out future data dependencies and abstracting logic into modular, O(1) lookup architecture designed for long-term scalability. This approach relies on a holistic, cognitive map of the entire project context. The capacity to perform such a "in the future" plan is hard for AI to consider, further pointing out context drift as a major problem in development.

Token Level Optimization is the “predicting of the most statistically probable sequence of code characters required to fulfill immediate, localized functional execution” (Harding 2025). This “optimization approach causes a significant spike in downstream software defects” (Harding 2025). Since the AI optimizes for immediate response tokens, it frequently generates monolithic, rigid, and high complexity O(n) or greater code that ignores broader system efficiency, future extensibility, and global maintainability. This short-sighted optimization was explicitly seen in the Simulated Tuning trials, where the models routinely stripped away comments and docstrings across prompts to optimize immediate token output, completely sacrificing the programs long-term readability.

In the evaluation section, the playability metrics followed a distinct bell curve. Playability generally improved from Zero-Shot to Single/Few-Shot, peaking at 6.33 for GPT 5.5, as the models received crucial thematic context (Figure 38). However, scores plummeted during Simulated Tuning, dropping to 3 for GPT 5.5. As the prompts became highly granular and the context window extended, the models lost track of the overarching gameplay loop, optimizing for requested micro-functions at the expense of general playability. The Human-AI Collaborative Game, scored an exemplary 6.5/7 in Playability, aligning with contemporary research surrounding Mixed-Initiative Co-Creativity (MI-CC). While LLMs excel at procedural content generation and fulfilling strict functional programming constraints, they lack an intrinsic understanding of Player Experience and emotional pacing. Humans possess the necessary "systems-level empathy" to establish what game design frameworks define as “Design Pillars,” the core experiential goals that dictate how a mechanic should feel to a user (Geheeb 2026). Recent studies confirm that while LLMs generate code efficiently, they struggle with "mutual non-contradiction" and cannot independently maintain a game's experiential goals over time (Geheeb 2026). The human

developer continuously enforces the design pillars, ensuring the generated mechanics structurally translated the thematic intent, effectively preventing the model from optimizing for short term functionality at the expense of engagement and long-term fun.

To validate the game evaluation statistics established in Figure 38, a comprehensive cross-examination was conducted against the aggregated empirical user data collected from anonymous play testers (Figures 31–37). The player data tracks six dimensions derived from Compton's evaluation rubric: Clarity, Flow, Balance, Length, Integration, and Fun, on a 7-point scale. The evaluation scored games 1 and 2, Zero-shot prompts of 4.3 and 5.5 respectively, with an average of 3 and 5.67. The play tester data strongly corroborates this evaluation where game 1 suffered across all playtesting metrics, averaging to 3.19 and game 2 averaged to 4.19. The play testers scored about the same for game 1 while scoring lower for game 2, showcasing that general users are more likely to score a game lower if it doesn't adhere to the exact constraints of the problem. The evaluation scored games 3 and 4, Single/Few-shot prompts of 4.3 and 5.5 respectively, with an average of 5.33 and 6.33. The play tester data scored these games much lower at 3.22 and 3.65 respectively, showcasing that the games did improve in playability, but still scores relatively lower compared to the evaluation section. The evaluation scored games 5 and 6, Simulated Tuning of 4.3 and 5.5 respectively, with an average of 3.33 and 3. The play tester data scored these games at 2.4 and 3.72 showing a little fluctuation compared to the scores during evaluation. Finally, game 7 during evaluation scored a 6.5 while the user data suggested a 4.66. The anonymous user data scored lower for game 7, however it still had the largest average out of the differing prompting strategies. Despite the difference in averages between evaluation and user study, GPT 5.5 remained dominant over GPT 4.3. The absolute gap closed slightly in the play tester data due to general user sensitivity to edge-case bugs, but the hierarchical relationship between the models remained

unchanged. The evaluation and the user data show a clear performance boost when transitioning from zero-shot to few-shot environments for both models, validating that strict prompt constraints successfully elevate the performance floor for lower-parameter models. However, the structural breakdown and performance regression identified during Simulated Tuning in both data sets reveal severe mechanical degradation, confirming that Large Language Models inevitably suffer from catastrophic context drift and hallucinations. Most critically, Game 7 represents the peak of both studies, securing the highest overall rating in the evaluation section and by play testers.

Conclusion

Large Language Models can be a tool of great use in tandem with a human being. We compared two models of ChatGPT, 4.3 and the recent 5.5. This comparison was to not only show the work product of the models across different prompting strategies to emulate a scrum framework of game design, but also to showcase the evolution and improvements between models. The difference in release of these models is little over a year, and in that time there has been improvement but the underlying problems: poor efficiency, shortsided maintainability, and hallucination, are all still very present in newer models. Although AI can try to make games, it fails to understand the core concept of what makes a game a game, nor does it have the intrinsic world knowledge of what mechanics and themes work well together in the video game market. Therefore, the notion that artificial intelligence is currently capable of wholly replacing human game designers is demonstrably false. The data collected across these case studies reveals a critical distinction between programmatic code generation and holistic game design. While LLMs excel at rapidly producing boilerplate scripts and functional prototypes under zero-shot constraints, their performance paradoxically degrades when subjected to extended, iterative feedback loops. Systemic vulnerabilities such as context drift, intrinsic hallucinations, and strict adherence to

token-level optimization prevent AI from maintaining a consistent "world state" or applying the systems-level architecture required for scalable software development. The superior performance of the Human-AI Collaborative Development trial, scoring the highest in both Code Quality and Playability, establishes the most effective method for modern game development. Ultimately, video games are not merely syntactically correct strings of logic, they are complex interdisciplinary experiences driven by interactive design and user psychology. Generative AI is an unprecedented utility for accelerating production pipelines, debugging, and rapid prototyping. However, the fundamental soul of a game, its pacing, its balance, and its capacity to actively engage a player remains a distinctly human ingenuity. Artificial intelligence will not replace the game designer, rather, it will serve as a foundational instrument for the designers who learn to orchestrate it effectively.

Works Cited

D. P. Kristiadi, F. Sudarto, D. Sugiarto, R. Sambera, H. L. H. S. Warnars and K. Hashimoto, "Game Development with Scrum methodology," *2019 International Congress on Applied Information Technology (AIT)*, Yogyakarta, Indonesia, 2019, pp. 1-6, doi: 10.1109/AIT49014.2019.9144963.

Harman, A. M. (2023). *Development Methodologies in the Game Development Industry* (Doctoral dissertation, Kutztown University of Pennsylvania).

Kramarzewski, A., & De Nucci, E. (2023). *Practical Game Design: A modern and comprehensive guide to video game design*. Packt Publishing Ltd.

Filipović, A. (2023). The role of artificial intelligence in video game development. *Kultura polisa*, *20*(3), 50-67.

Treanor, M., Zook, A., Eladhari, M. P., Togelius, J., Smith, G., Cook, M., ... & Smith, A. (2015). AI-based game design patterns.

Riedl, M. O., & Zook, A. (2013, August). AI for game production. In *2013 IEEE conference on computational intelligence in games (CIG)* (pp. 1-8). IEEE.

Heston, T. F., & Khun, C. (2023). Prompt engineering in medical education. *International Medical Education*, *2*(3), 198-205.

Velásquez-Henao, J. D., Franco-Cardona, C. J., & Cadavid-Higuita, L. (2023). Prompt Engineering: a methodology for optimizing interactions with AI-Language Models in the field of engineering. *Dyna*, *90*(SPE230), 9-17.

Federiakin, D., Molerov, D., Zlatkin-Troitschanskaia, O., & Maur, A. (2024, November). Prompt engineering as a new 21st century skill. In *Frontiers in Education* (Vol. 9, p. 1366434). Frontiers Media SA.

Esposito, N. (2005, January). A short and simple definition of what a videogame is. In *Proceedings of DiGRA 2005 Conference: Changing Views: Worlds in Play*.

Tavinor, G. (2008). Definition of videogames. *Contemporary Aesthetics (Journal Archive)*, *6*(1), 16.

Baer, R. H. (2010). *The medium of the video game*. University of Texas press.

Van Rossum, G., & Drake, F. L. (2003). *An introduction to Python* (p. 115). Bristol: Network Theory Ltd..

Patel, A., Sultana, K. Z., & Samanthula, B. K. (2024, December). A Comparative Analysis between AI Generated Code and Human Written Code: A Preliminary Study. In *2024 IEEE International Conference on Big Data (BigData)* (pp. 7521-7529). IEEE.

Bayram, A., Menekse Dalveren, G. G., & Derawi, M. (2025). Comparative Analysis of AI Models for Python Code Generation: A HumanEval Benchmark Study. *Applied Sciences*, *15*(18), 9907.

McAllister, G., & White, G. R. (2015). Video game development and user experience. In *Game user experience evaluation* (pp. 11-35). Cham: Springer International Publishing.

Methodology. (n.d.). In *Merriam-Webster.com dictionary*. Retrieved March 10, 2026, from https://www.merriam-webster.com/dictionary/methodology

PM , Agile vs Waterfall Model: Key Differences Between Waterfall and Agile, Agile vs, and the Waterfall Approach. (2025, May 26). Project Management Course. https://projectmgmtcourse.com/agile-vs-waterfall-model/

Schwaber, K., & Sutherland, J. (2020, November). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. https://scrumguides.org/scrum-guide.html

Scrum.org. (2026). *What is Scrum?* Scrum.org. https://www.scrum.org/learning-series/what-is-scrum/

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. (2023). *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. https://arxiv.org/pdf/2302.11382

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877-1901.

Liu, P., Zhang, L., & Gulla, J. A. (2023). Pre-train, Prompt, and Recommendation: A Comprehensive Survey of Language Modeling Paradigm Adaptations in Recommender Systems. *Transactions of the Association for Computational Linguistics*, *11*, 1553–1571. https://doi.org/10.1162/tacl_a_00619

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023, May 25). *Self-Refine: Iterative Refinement with Self-Feedback*. ArXiv.org. https://doi.org/10.48550/arXiv.2303.17651

Capitol Technology University. (2025, October 23). *AI in video game development: From smarter NPCs to procedural worlds*. Capitol Technology University Blog. https://www.captechu.edu/blog/ai-in-video-game-development

Friedman, N. (2021). *Introducing GitHub Copilot: Your AI pair programmer.* (Primary source for the "Pair Programmer" nomenclature).

Dohmke, T., Iansiti, M., & Levien, G. (2023). *Sea Change in Software Development: Economic and Productivity Analysis of the AI-Powered Developer.* (Often cited regarding the 55% speed increase in developers using Copilot).

Compton, M. (n.d.). *Game evaluation criteria (5 minutes or less)* [PDF]. University of New Mexico Gamification Resources. https://www.unm.edu/~unmvclib/gamification/assessment/gameevaluationcriteria.pdf

Gunter, G. A., Kenny, R. F., & Vick, E. H. (2008). A Case for a Formal Design Paradigm for Serious Games. *Journal of Educational Technology Systems*, *37*(1), 85–112. https://doi.org/10.2190/ET.37.1.c

Johnson, P. (2018). *Fundamentals of Collection Development and Management* (4th ed.). ALA Editions.

Tulqin o‘g’li, M. T. O. (2025). Rubric design for grading programming assignments: Ensuring objectivity. *International Journal of Academic Engineering Research*, *9*(9), 43–48. IJAER250907.pdf

Wang, Z., et al. (2026). Beyond Correctness: Benchmarking Multi-dimensional Code Generation for Large Language Models. *OpenReview*.

ChromaDB: Context Rot: Evaluating LLM Performance Degradation with Increasing Input Tokens - ZenML LLMOps Database. (2025). Zenml.io. https://www.zenml.io/llmops-database/context-rot-evaluating-llm-performance-degradation-with-increasing-input-tokens

Ziwei, et al. *"Survey of Hallucination in Natural Language Generation."* ACM Computing Surveys, vol. 55, no. 12, March 2023, Article No. 248.

Harding (2025) *"AI Copilot Code Quality: Evaluating 2025's Increased Defect Rate with Data"* GitClear 2025 Report

Brown, T et al. (2020). *Language Models are Few-Shot Learners*. https://arxiv.org/pdf/2005.14165

Mohammad Khalid et al (2025). ERGO: Entropy-guided resetting for generation optimization in multi-turn language models. In *Proceedings of the 2nd Workshop on Uncertainty-Aware NLP (UncertaiNLP 2025)* (pp. 273–286). Association for Computational Linguistics. ERGO: Entropy-guided Resetting for Generation Optimization in Multi-turn Language Models

Jackson, et al. (2026, July). The role of LLMs in collaborative software design. In *Proceedings of the 34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (pp. 1–10). Association for Computing Machinery.The Role of LLMs in Collaborative Software Design

Dongre, V., Rossi, R. A., Lai, V. D., Yoon, D. S., Hakkani-Tür, D., & Bui, T. (2025). *Drift no more? Context equilibria in multi-turn LLM interactions* (arXiv:2510.07777v1). arXiv. Drift No More? Context Equilibria in Multi-Turn LLM Interactions

Geheeb, J., Schwarz, M. J., Dyrda, D., & Groh, G. (2026, August). LLMs are the ideal candidate for mixed-initiative game design pillar workflows. In *Proceedings of the 2026 International Conference on the Foundations of Digital Games (FDG '26)*. Association for Computing Machinery. [2605.09767] LLMs are the Ideal Candidate for Mixed-Initiative Game Design Pillar Workflows

Appendix

Zero shot prompt:

I want you to make me a game in python about grape picking.

Single/Few Shot Prompt

I'm going to take you through the game development process to develop a game using python and the pygame library. To start our idea will be a game about "grape picking." We will be developing this game as a text based game with a simple graphical interface. Our game will look similar to that of the Oregon trail game where a person can choose from multiple options to certain scenarios and progress through the story to eventually survive. Certain options will lead to random events that could cause a game over for the player. For simplicity we will start with a version of the game that creates a GUI that is just text and buttons that represent options. Remember to stay on theme about grape picking, which can then lead to some adventure. I need you to generate all python files that will be helpful in displaying the gui, all the text, and all paths that the game can follow. An example of how one would do something like this is a series of if statements that create branching paths and display text based on what path we are in.

Simulated Tuning prompts:

We are at the concept and ideation stage of game development, I want you to discuss with me ideas about how to make a game about grape picking.

We are now in stage 2 of game development: Assets and Environment creation. I want you to think about and implement some assets or environments that you can use to create this game revolving around the ideas you produced in step 1

We are now in stage 3: character and NPC Behavior, I want you to think about how you could implement entities, NPCs, and emotional modeling for ideas discussed in the previous stages

We are now in stage 4: Dialogue, narrative and writing, I want you to create a story system befit of the ideas discussed in previous sections

We are now in stage 5: Gameplay mechanics and system design. I want you to think about how you can create gameplay from the previous sections.

We are now at the implementation stage. Implement the game as you see fit including any files, scripting, assets you desire.

We are now in stage 7: testing , QA, and Optimization. Play testers have stated that the dialogue and story promised has not been delivered.

Survey Form:

https://forms.gle/1EnRno66pL3nKcMPA

Project Repository:

Brock555555/Ai_Thesis_Project.